

Resume-to-Job-Description Fit Scorer — Build Explanation

1. Design parameter — scoring weights. I chose a configurable weighted model of Skills 40%, Experience 20%, Education 15%, Responsibilities 15%, and Tools 10%. Skills receives the largest weight because a direct technical mismatch is usually more consequential for a junior developer role than a secondary tool mismatch. The values live in `app/config.py`, so the scoring policy can change without editing scoring logic.

2. Failure observed. An invalid PDF byte stream caused PDF extraction to fail because the file was not a valid PDF. I changed the parser to catch parsing exceptions and return a structured warning. For image-only PDFs, near-empty extracted text is also detected and reported as an OCR requirement rather than fabricating evidence.

3. Metric tracked. I tracked end-to-end scoring latency in milliseconds and expose it as `latency_ms` in every response. This gives a concrete baseline for comparing the LLM path with deterministic fallback. I also included a consistency check: two identical resumes should produce the same score, while a third materially different sample is included for calibration discussion.

4. Not finished / next step. I did not build OCR or a recruiter-labeled evaluation dataset. Next I would collect anonymized JD/resume pairs with recruiter labels, measure agreement and score variance on a held-out set, tune weights/thresholds, and add OCR for scanned PDFs. The implementation is explicit about these limitations rather than presenting keyword fallback as production-grade semantic matching.